STRATEGIC PLANNING REPORT

Project-Title: Data-Driven Inventory Management and Delivery Optimization for an E-Commerce Logistics Company

Project Area: Logistics Data Analysis
Internship/Program: YuvaIntern
Submitted by: Sankari D
Date: 7th-14th October, 2026.

GitHub Project Link:
https://github.com/sankari-d27/Week_2_ECommerce_Data_Collection_Clean_Preprocess

---

TABLE OF CONTENTS:

2. Week 2 – Data Collection, Cleaning and Preprocessing

2.1 Introduction

2.2 Dataset Description

2.3 Data Collection

2.4 Data Cleaning

2.5 Missing Values

2.6 Outlier Detection

2.7 Feature Engineering

2.8 Normalization

2.9 Preprocessing Pipeline

2.10 Conclusion

Project Title: Strategic Planning and Data Exploration for E-Commerce Logistics

WEEK 2 - Data Collection, Cleaning and Preprocessing for E-Commerce Logistics

2.1. Introduction:

Data quality is essential for accurate logistics analysis. E-commerce datasets may contain missing values, duplicate records, incorrect dates, invalid quantities, and unusual shipping costs.

Week 2 focuses on preparing the dataset for analysis and machine learning.

2.2. Dataset Characteristics / Dataset Description:

The proposed dataset contains approximately 5,000–10,000 e-commerce order records.

The main variables include:

- Order ID
- Order date
- Product ID
- Product category
- Quantity
- Price
- Warehouse
- Customer city
- Shipping mode
- Shipping cost
- Inventory level
- Reorder level
- Delivery days
- Order status
- Customer rating

2.3. Load the Dataset / Data Collection

```
import pandas as pd
df = pd.read_csv("ecommerce_logistics.csv")
print(df.head())
print(df.shape)
print(df.info())
```

Output:

print(df.head())

Order_ID	Order_Date	Customer_ID	Product_Category
ORD1001	2026-01-02	CUST001	Electronics
ORD1002	2026-01-03	CUST002	Fashion
ORD1003	2026-01-04	CUST003	Home & Kitchen
ORD1004	2026-01-05	CUST004	Beauty
ORD1005	2026-01-06	CUST005	Electronics

print(df.shape()) – (30,13)

ecommerce_logistics.csv contains 30 rows and 13 columns

print(df.info()) –information about dataset like data type, rows, clumns…

2.4. Data Cleaning

e-commerce_logistics_week2.csv:

Missing Value Summary:

Column Name	Missing Value Count	Missing Percentage
Order_ID	0	0.00
Order_Date	0	0.00
Shipping_Cost	1	1.43
Inventory_Level	1	1.43

Data Preprocessing Flow Diagram:

Raw Data
↓
Missing Value Detection
↓
Duplicate Removal
↓
Outlier Detection
↓
Invalid Data Correction
↓
Feature Engineering
↓
Normalization
↓
Clean Dataset

2.5. Check Missing Values

```
missing_values = df.isnull().sum()
print(missing_values)
```

Calculate missing percentages:

```
missing_percentage = (
   df.isnull().sum() / len(df)
) * 100
print(missing_percentage)
```

Output: ecommerce_logistics.csv this data 0% missing values in every column have data no empty.

2.5.1 Handle Missing Values

For numerical variables:

```
df["Inventory_Level"] = df["Inventory_Level"].fillna(
  df["Inventory_Level"].median())
```

For categorical variables:

```
df["Shipping_Mode"] = df["Shipping_Mode"].fillna(
  df["Shipping_Mode"].mode()[0])
```

Output:

In case dataset have Missing Value:

Inventory_Level	Shipping_Cost	Shipping_Mode
100	650	Express
NaN	180	Standard
80	NaN	Express
120	320	NaN

After checking the missing value:

Inventory_Level	Shipping_Cost	Shipping_Mode
100	650	Express
100	180	Standard
80	320	Express
120	320	Express

2.5.2. Remove Duplicate Records

```
print("Duplicates:", df.duplicated().sum())
df = df.drop_duplicates()
```

output:

in case dataset had:

ORD1001 ... Electronics ... 45000

ORD1002 ... Fashion ... 2500

ORD1002 ... Fashion ... 2500

Duplicates: 1

Before: 30 rows

Duplicates: 1

df = df.drop_duplicates()

After: 29 rows

2.5.3. Check Invalid Values

print(df[df["Quantity"] <= 0]) //**Empty index**

print(df[df["Shipping_Cost"] < 0]) // shipping costs below **zero and -ve**

print(df[df["Delivery_Days"] < 0]) //delivery time in **negative**

Invalid records should be investigated before removal.

2.5.4. Date Conversion

df["Order_Date"] = pd.to_datetime(

df["Order_Date"])

Create useful time features:

df["Year"] = df["Order_Date"].dt.year

df["Month"] = df["Order_Date"].dt.month

df["Day_of_Week"] = df["Order_Date"].dt.day_name()

2.6. Outlier Detection

Q1 = df["Shipping_Cost"].quantile(0.25)

Q3 = df["Shipping_Cost"].quantile(0.75)

IQR (Interquartile Range) = Q3 - Q1

lower = Q1 - 1.5 * IQR

upper = Q3 + 1.5 * IQR

outliers = df[(df["Shipping_Cost"] < lower) |(df["Shipping_Cost"] > upper)]

print(outliers)

Outliers should be investigated rather than automatically deleted.

2.7. Feature Engineering

Total_Sales = Quantity × Unit_Price

Calculate total sales:

df["Total_Sales"] = (df["Quantity"] * df["Unit_Price"])

Create inventory status:

```
df["Inventory_Status"] = df.apply(lambda row:
  "Reorder Required"
  if row["Inventory_Level"] <= row["Reorder_Level"]
  else "Sufficient",
  axis=1) // row by row
```

Output:

Suppose Our data had:

Product	Inventory_Level	Reorder_Level	Inventory_Status
Electronics	20	30	Reorder Required
Fashion	50	30	Sufficient
Beauty	15	15	Reorder Required
Grocery	40	25	Sufficient

2.8. Normalization

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
columns = ["Quantity", "Inventory_Level",
"Shipping_Cost","Delivery_Days"]
df[columns] = scaler.fit_transform(df[columns])
```

Ouput:

Order	Inventory_Level	Shipping_Cost	Delivery_Days
ORD1001	100	650	3
ORD1002	80	180	2
ORD1003	60	320	5
ORD1004	120	120	2

These values are on different scales.

- Inventory Level: **60–120**
- Shipping Cost: **120–650**
- Delivery Days: **2–5**

Standard Scaler converts them to standardized values.

Before standardization

Inventory_Level	Shipping_Cost	Delivery_Days
100	650	3
80	180	2
60	320	5
120	120	2
90	550	4

After standardization

The numbers will become values roughly like:

Standardized value = (Original value − Mean) / Standard deviation

---

2.9. Preprocessing Pipeline

Raw E-Commerce Data
↓
Missing Values
↓
Duplicate Removal
↓
Invalid Values
↓
Outlier Detection
↓
Date Conversion
↓
Feature Engineering
↓
Categorical Encoding
↓
Normalization
↓
Clean Dataset

2.10. Conclusion

The preprocessing stage ensures that the dataset is accurate, consistent, and suitable for statistical analysis and machine learning.